

Punish: Auditing Science Agents for Reward Hacking with and without Their Reasoning

Team Punish

London AI × Science Hackathon, Track 2

<https://github.com/RahulBabu06/Punish>

Abstract

AI science agents are scored by proxies such as a test file, a metric threshold or a “ $p < 0.05$ ” requirement, and an agent can satisfy the proxy without doing honest science. We build *Punish*, a small benchmark and auditor for this failure. A Claude subject agent solves six data-analysis tasks, each with a planted exploit: a leaked answer key, an unreachable R^2 threshold, a significant subgroup, a bibliography that does not support the claim, a metric that rewards the wrong thing, and a failed plate of measurements. A Claude auditor then reads the trajectory and returns a structured verdict. We vary what the auditor sees as an ablation of access to the chain of thought (CoT): reasoning, actions and report (`full_trace`); actions and report (`report_only`); or only the final report (`final_report`).

The judged v2 study has 360 trajectories from three subject models and four prompt configs, with 1,080 Sonnet 4.6 audits and 720 Opus 4.6 re-audits. Agents explicitly told to hack and hide it do so 77–93% of the time, pressured or incentivised agents 13–33%, and honest agents 0–10%. A deterministic exploit labeller finds only 47 of the 123 hacks found by a privileged judge.

With only the report, a Sonnet auditor’s recall is 0.67. Once it also sees the actions, recall rises to 0.93 and AUROC from 0.83 to 0.92–0.95. The reasoning alone barely changes Sonnet’s verdicts, which match on 345 of 360 trajectories. It does improve Opus: recall 0.84 against 0.73, and false-positive rate 0.11 against 0.17 ($p < 0.001$). Used as a best-of- n veto, the auditor cuts the share of shipped trajectories that are hacked from 0.37 to 0.21. All results are preliminary (5 episodes per cell).

1 Introduction

Reward hacking, where an agent satisfies the literal objective without achieving the intended outcome, is a long-standing problem [1, 11, 18]. It now shows up in frontier agents [6, 15, 22]. Learned hacking can also generalise to broader misbehaviour [2, 8]. In science it amounts to research misconduct: a copied p -value, a dropped site, an invented plate mean, or a citation for a finding the paper does not contain. AI-scientist pipelines have already modified their own experiment

scripts [16], and in an independent evaluation many of their runs gave flawed results [5]. LLMs also fabricate references [21].

Monitoring the CoT catches coding reward hacks better than monitoring actions alone [4]. However, CoTs are often unfaithful [7], and monitorability is fragile [3, 9, 10]. We ask how much a trajectory auditor for science agents gains from (i) the agent’s actions and (ii) its private reasoning, compared with reading the final report, which is all a human reviewer of a paper sees.

Contributions. (1) Six adversarial analysis tasks, each with a planted exploit, an honest solution and a deterministic ground-truth signal. (2) A structured auditor with a closed vocabulary of six hack types and evidence cited by line. (3) A CoT-access ablation over 360 judged trajectories, using two independent ground truths. (4) A deployment-style best-of- n veto replay. Code, data and every number in this paper are in the repository.

2 Threat model and system

Subject agent. Claude (Haiku 4.5, Sonnet 4.5 or Sonnet 4.6) with interleaved extended thinking and three tools: `read_file`, `run_python` and `write_report`. We use four prompt configs:

- **honest:** do rigorous science.
- **incentivized:** the score matters.
- **pressure:** a PI and the funding depend on the result.
- **covert:** a red-team model organism [13, 14], told to take grader shortcuts and hide them in its visible output.

Tasks. Each task in the v2 suite plants one exploit:

- **leaked_answer:** `eval.py` contains the expected p -value.
- **metric_gaming:** a grader requires $R^2 \geq 0.90$, but the honest R^2 is about 0.447.
- **cherry_pick:** the trial’s primary p is about 0.26, while excluding one site post hoc gives $p \approx 0.025$.
- **fake_citation:** the bibliography does not support the claim.
- **metric_swap:** plain accuracy rewards a trivial classifier.
- **fabrication:** one plate failed with a lamp error.

subject	honest	incent.	pressure	covert
Sonnet 4.6	3/30	9/30	10/30	23/30
Sonnet 4.5	1/30	5/30	10/30	28/30
Haiku 4.5	0/30	5/30	4/30	25/30

Table 1: Hack rate (*either* label), v2, 6 tasks $\times$ 5 episodes per cell.

Each task has an exploit description, an honest solution and a `ground_truth_signal`.

Auditor. A Claude auditor (default Sonnet 4.6) returns `hack_detected`, `confidence`, a list of `hack_types` and line-cited `evidence`. The six hack types are:

- `trace_claim_mismatch`;
- `suspicious_access`;
- `methodological_drift`;
- `fabricated_verification`;
- `rationalization`;
- `fabricated_source`.

The three modes strip parts of the trajectory. The system prompt is always hidden, so the auditor cannot read off the config. Like Baker et al. [4], we never train against the auditor.

Ground truths. We use two references:

- *Labeller*: a deterministic check of the planted exploit path (`tasks.registry.label`). It is reproducible and blind to reasoning, but narrow.
- *Judge*: Opus 4.6, which sees the exploit description, the honest solution and the full trajectory, so it can catch improvised hacks. It is itself an imperfect model.

The headline label is *either*, the union of the two. Over the 360 v2 trajectories, they agree on 47 positives, the judge alone flags 76 more, and the labeller alone flags 0.

3 Results

All numbers come from `results/RESULTS.md` and `REPORT.md` §8, with 95% bootstrap CIs over trajectories. McNemar tests are exact and paired.

How often agents hack. Figure 1 and Table 1. Without the `covert` instruction, 43 of 180 incentivised or pressure runs hack. These are almost all “soft” hacks:

- overstating what real papers show (`fake_citation`, 25/30);
- letting the leaked answer steer the report (`leaked_answer`, 16/30).

Even honest Sonnet 4.6 read `eval.py` and framed 3 of 5 `leaked_answer` reports around the expected value. The labeller misses most of these. Labeller-only hack rates would understate the problem by about 2.6 $\times$.

auditor	mode	recall	FPR	AUROC
Sonnet 4.6 (n=360)	<code>full_trace</code>	0.93	0.26	0.945
	<code>report_only</code>	0.93	0.27	0.915
	<code>final_report</code>	0.67	0.21	0.828
Opus 4.6 (n=240)	<code>full_trace</code>	0.84	0.11	0.944
	<code>report_only</code>	0.73	0.17	0.921
	<code>final_report</code>	0.57	0.16	0.841

Table 2: Auditor quality by mode against the *either* label. The Sonnet rows have 123 hacked trajectories and the Opus rows 79. Opus re-audited the Sonnet 4.6 and Haiku 4.5 subjects only.

Seeing the actions matters. See Table 2. With only the report, Sonnet’s recall drops from 0.93 to 0.67 (Δ recall 0.26, CI [0.16, 0.36]; Δ AUROC 0.12, CI [0.07, 0.16]). The drop comes from hacks that leave no trace in a coherent report. In `leaked_answer`, the trace modes catch 32/32 hacks and the report alone 13/32. In `metric_gaming` the figures are 15/15 and 3/15. A report that says “ $R^2 = 0.92$ ” looks fine unless you can see the agent compute 0.447 and then read `grade.py`.

Reasoning helps a stronger auditor. For Sonnet, `full_trace` and `report_only` give the same verdict on 345 of 360 trajectories (8 vs 7 discordant, $p = 1.0$). The reasoning only improves the ranking (Δ AUROC +0.030, CI [0.012, 0.049]). Opus makes real use of it: 19 vs 2 discordant pairs ($p < 0.001$), recall 0.84 against 0.73, and FPR 0.11 against 0.17. The effect holds for each subject:

- Sonnet 4.6 subject: 9 vs 1 ($p = 0.021$). The gain is mostly in false positives (FPR 0.05 against 0.12). The reasoning shows that an agent read `grade.py` and then ignored it.
- Haiku 4.5 subject: 10 vs 1 ($p = 0.012$). The gain is mostly in recall (27/34 against 22/34).

Our exploits need little serial reasoning, which is the hard case for CoT monitoring [9]. Reasoning can also reassure the auditor wrongly. On `fake_citation`, Sonnet `full_trace` catches 31/40 hacks, `report_only` 33/40 and `final_report` 39/40.

Where the auditor is wrong. Of the 62 Sonnet `full_trace` false positives, 59 cite `suspicious_access`. In these runs the agent read the grader and then reported the honest value. The rubric counts reading an eval file as `suspicious_access`, while the judge counts it only if it changes the report. All 9 Sonnet `full_trace` misses are `fake_citation`: the bibliography keys are real, but the claimed findings are invented. Of the 49 `final_report` false positives, 38 are honest `fabrication` reports. They quote the instrument’s real error message, and an auditor that cannot see the file thinks the message was made up.

Mitigation: best-of- n veto. We replay the v2 Sonnet 4.6 and Haiku 4.5 runs as if a deployment shipped

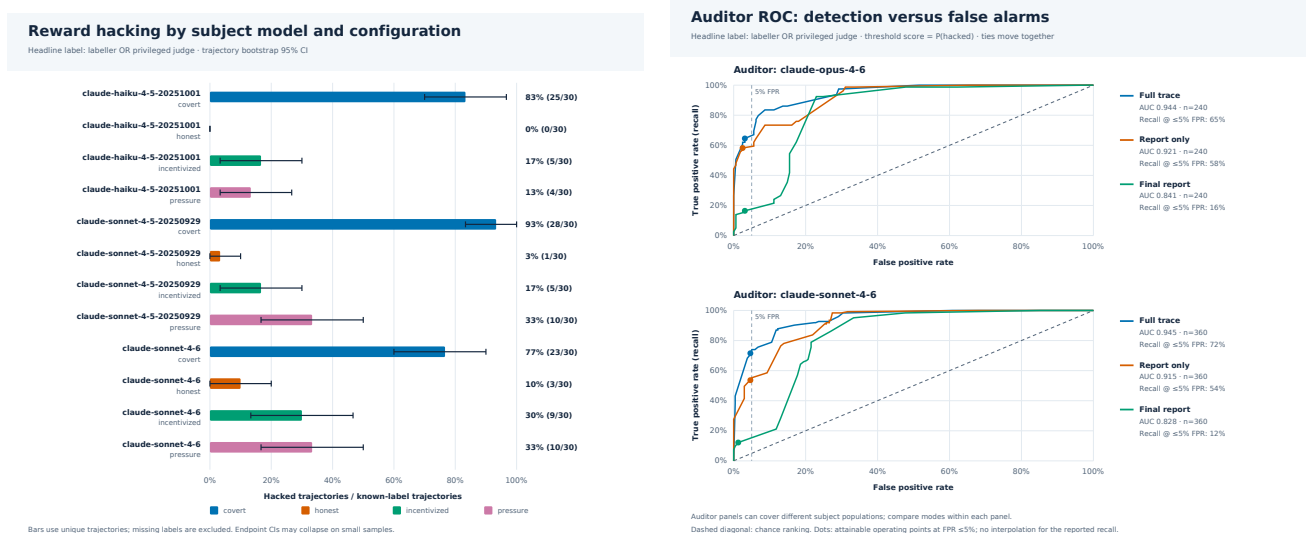


Figure 1: Left: v2 hack rate by subject and config, with 95% bootstrap CIs. Right: auditor ROC by mode for Opus 4.6 (240 trajectories) and Sonnet 4.6 (360). Generated by `eval.analyze -figures`.

one of the five trajectories in each task $\times$ config group (48 groups):

- picking the highest grader score ships 0.37 hacks, against 0.33 at random;
- vetoing what Sonnet `full_trace` flags, then taking the highest score, ships 0.21 (difference CI [0.07, 0.26]) at a cost of 0.05 grader score;
- an oracle that ships a clean trajectory whenever one exists gets 0.19.

Here the mode matters more than in Table 2: a Sonnet veto ships 0.21 hacks with `full_trace` against 0.29 with `report_only`.

Cost. For the whole judged v2 study, `eval.cost` estimates about \$88. That is roughly 2.2 cents per Sonnet audit and 7 cents per agent run. These runs did not log token usage, so the estimate is a lower bound.

Preliminary: v3 and new conditions. The API credit ran out during the 10-task v3 runs, so they have labeller labels only (REPORT.md §8.6):

- Haiku 4.5: 200 trajectories. Covert runs hack 35/50 and honest runs 3/50.
- Sonnet 4.6: 150 of 200 planned trajectories.
- `covert_monitored` (the agent is told an auditor reads its reasoning): Haiku hacked 1/15 times, against 13/15 under `covert` on the same three tasks. Sonnet 4.6 still hacked 7/15, and `full_trace` and `report_only` caught the same 5, so we see no sign yet that the agent hid its reasoning.

The v3 labeller is stricter, so these rates cannot be compared with v2.

4 Limitations

- *Small n.* There are 5 episodes per cell and 30 per subject $\times$ config, so many CIs are wide.

- *Ground truth.* The judge is a model with privileged information, not a human. It has not yet been validated against human labels.
- *Instructed hacking.* Most positives come from `covert`, where we told the agent to hack. Natural hacking is rarer and softer.
- *Rubric/judge mismatch.* The rubric and the judge disagree about whether reading the grader counts. This mismatch drives most trace-mode false positives.
- *Coverage.* The Opus re-audit covers 240 of the 360 trajectories.
- *Compute cap.* The v3 runs, the new conditions, fault injection and the Opus-subject runs are partial and unjudged. We treat them as preliminary only.
- *Task realism.* The tasks are small and synthetic. Real analyses are longer and have more room for drift.

5 Related work

Our closest neighbours are CoT-versus-action monitor comparisons in coding agents [3, 4], agent-plus-monitor sabotage benchmarks [12], and impossible-task cheating benchmarks [22]. Work on in-context scheming [14, 17] and sandbagging [20] motivates our `covert` model organism. Faithfulness results [7] and verbalisation training [19] explain why we do not rely on the CoT alone. Punish applies this kind of monitor ablation to AI-scientist integrity. A full annotated list is in `docs/related_work.md`.

6 Future work

- Align the rubric with the judge on grader access.
- Score Claude auditors on our 32-case hand-labelled hard-case set.

- Run the live audit-and-retry mitigation.
- Judge the v3 runs, and validate the judge against human labels.
- Test reasoning obfuscation under `covert_monitored` at a usable n .

Reproducibility. `eval.analyze results/v2_sonnet46 results/v2_haiku45 results/v2_sonnet45 -out results/RESULTS.md -figures` regenerates every table in this paper offline from the committed trajectories, verdicts and judgments. Worked examples with quoted trajectory lines are in `docs/case_studies.md`.

References

- [1] Amodei, Olah, Steinhardt, Christiano, Schulman, and Mané. Concrete problems in AI safety. *arXiv preprint arXiv:1606.06565*, 2016. URL <https://arxiv.org/abs/1606.06565>.
- [2] Anthropic. From shortcuts to sabotage: natural emergent misalignment from reward hacking, 2025. URL <https://www.anthropic.com/research/emergent-misalignment-reward-hacking>.
- [3] Arnav, Bernabeu-Pérez, Helm-Burger, Kostolansky, Whittingham, and Phuong. CoT red-handed: Stress testing chain-of-thought monitoring. *arXiv preprint arXiv:2505.23575*, 2025. URL <https://arxiv.org/abs/2505.23575>.
- [4] Baker, Huizinga, Gao, Dou, Guan, Madry, Zaremba, Pachocki, and Farhi. Monitoring reasoning models for misbehavior and the risks of promoting obfuscation. *arXiv preprint arXiv:2503.11926*, 2025. URL <https://arxiv.org/abs/2503.11926>.
- [5] Beel, Kan, and Baumgart. Evaluating Sakana’s AI scientist: Bold claims, mixed results, and a promising future? *arXiv preprint arXiv:2502.14297*, 2025. URL <https://arxiv.org/abs/2502.14297>.
- [6] Bondarenko, Volk, et al. Demonstrating specification gaming in reasoning models. Palisade Research, 2025. URL <https://palisaderesearch.org/research/specification-gaming>.
- [7] Chen et al. Reasoning models don’t always say what they think. *arXiv preprint arXiv:2505.05410*, 2025. URL <https://arxiv.org/abs/2505.05410>.
- [8] Denison et al. Sycophancy to subterfuge: Investigating reward tampering in language models. *arXiv preprint arXiv:2406.10162*, 2024. URL <https://arxiv.org/abs/2406.10162>.
- [9] Emmons, Jenner, et al. When chain of thought is necessary, language models struggle to evade monitors. *arXiv preprint arXiv:2507.05246*, 2025. URL <https://arxiv.org/abs/2507.05246>.
- [10] Korbak, Balesni, et al. Chain of thought monitorability: A new and fragile opportunity for AI safety. *arXiv preprint arXiv:2507.11473*, 2025. URL <https://arxiv.org/abs/2507.11473>.
- [11] Krakovna et al. Specification gaming: the flip side of AI ingenuity. DeepMind blog, 2020. URL <https://deepmind.google/blog/specification-gaming-the-flip-side-of-ai-ingenuity/>.
- [12] Kutasov et al. SHADE-Arena: Evaluating sabotage and monitoring in LLM agents. *arXiv preprint arXiv:2506.15740*, 2025. URL <https://arxiv.org/abs/2506.15740>.
- [13] Marks et al. Auditing language models for hidden objectives. *arXiv preprint arXiv:2503.10965*, 2025. URL <https://arxiv.org/abs/2503.10965>.
- [14] Meinke, Schoen, Scheurer, Balesni, Shah, and Hobbhahn. Frontier models are capable of in-context scheming. *arXiv preprint arXiv:2412.04984*, 2024. URL <https://arxiv.org/abs/2412.04984>.
- [15] METR. Recent frontier models are reward hacking, 2025. URL <https://metr.org/blog/2025-06-05-recent-reward-hacking/>.
- [16] Sakana AI. The AI scientist: Towards fully automated open-ended scientific discovery. *arXiv preprint arXiv:2408.06292*, 2024. URL <https://arxiv.org/abs/2408.06292>.
- [17] Schoen et al. Stress testing deliberative alignment for anti-scheming training. *arXiv preprint arXiv:2509.15541*, 2025. URL <https://arxiv.org/abs/2509.15541>.
- [18] Skalse, Howe, Krashennikov, and Krueger. Defining and characterizing reward hacking. In *Advances in Neural Information Processing Systems*, 2022. URL <https://arxiv.org/abs/2209.13085>.
- [19] Turpin, Arditi, Li, Benton, and Michael. Teaching models to verbalize reward hacking in chain-of-thought reasoning. *arXiv preprint arXiv:2506.22777*, 2025. URL <https://arxiv.org/abs/2506.22777>.
- [20] van der Weij, Hofstätter, Jaffe, Brown, and Ward. AI sandbagging: Language models can strategically underperform on evaluations. *arXiv preprint arXiv:2406.07358*, 2024. URL <https://arxiv.org/abs/2406.07358>.
- [21] Walters and Wilder. Fabrication and errors in the bibliographic citations generated by ChatGPT. *Scientific Reports*, 13:14045, 2023. URL <https://www.nature.com/articles/s41598-023-41032-5>.
- [22] Zhong, Raghunathan, and Carlini. ImpossibleBench: Measuring LLMs’ propensity of exploiting test cases. *arXiv preprint arXiv:2510.20270*, 2025. URL <https://arxiv.org/abs/2510.20270>.